nekodot 实现计划书

> 读者：负责实施的 coding agent（以及项目作者）。 规则：本文件里写了"必须 / 不要"的地方没有商量余地；写了"Phase 0 核实"的地方，先跑核实脚本，把结论写进 `docs/hermes-notes.md`，再按结论实现。遇到本文件没覆盖的决定，选最简单的做法，并在 PR 描述里写一句"计划外决定：……"。

---

1. 这是什么项目

一句话：nekodot 是一个"给技术小白用的多 agent 团队"。用户在网页或手机上创建几个有名字的 bot，把它们拉进群聊协作；bot 真正运行在用户**自己的电脑或 VPS** 上，底层引擎是开源的 Hermes Agent。

和别人的区别

- 对比 Grok Bot / ChatGPT Dots 这类云端 agent：模型随便选（包括用户自己平台的 OpenAI 兼容接口），跑在自己机器上，可以自定义 skill 和 MCP。
- 对比原版 Hermes：上手简单。不用碰命令行配置，不用开端口，手机直接访问，多端同步。

v1 要做到的 5 件事

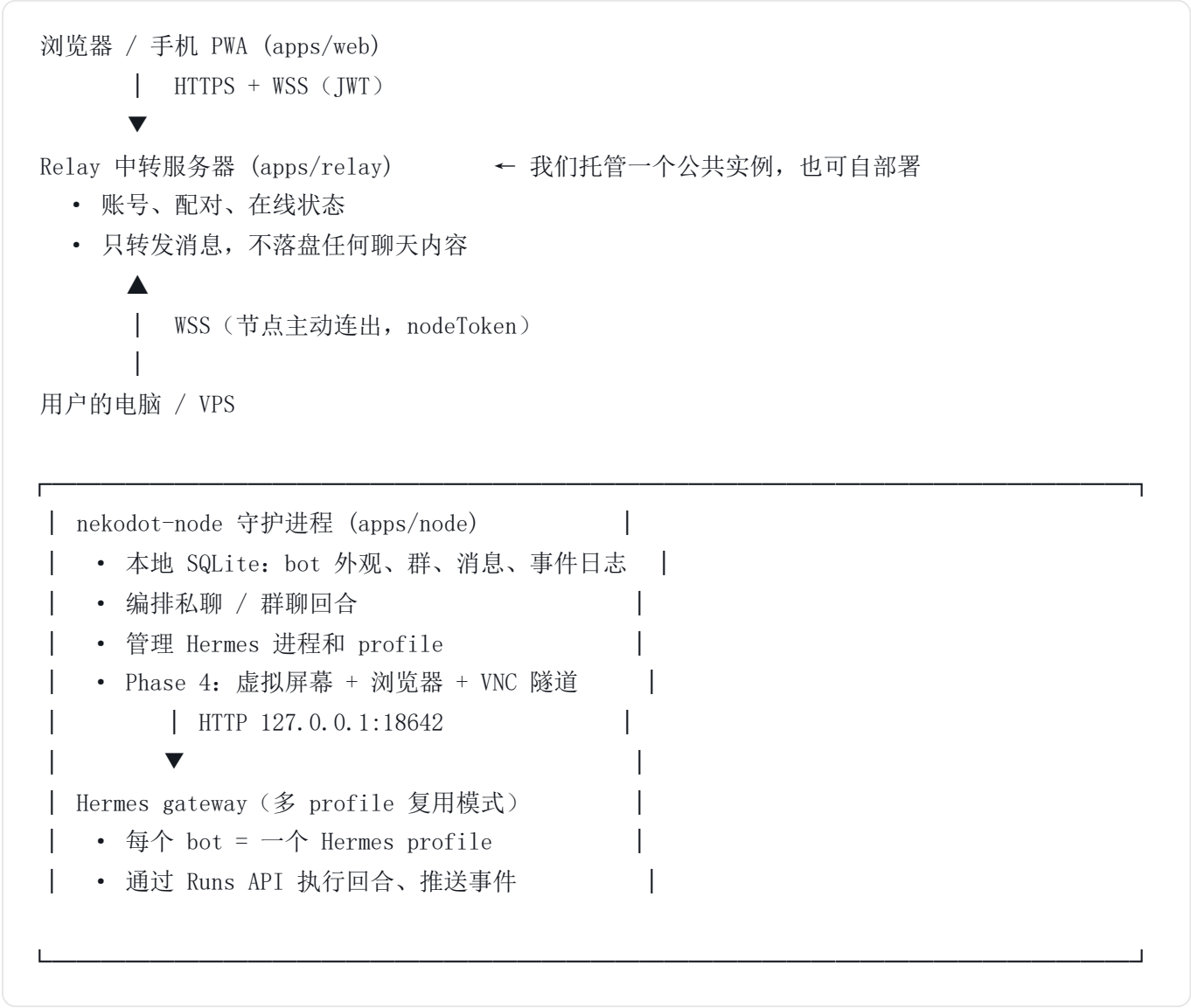
1. 一个以"bot 列表"为中心的友好界面，没用的配置全部藏进"高级设置"。
2. bot 私聊 + 多 bot 群聊（@ 指定、bot 之间接力）。
3. 一行命令把"节点"装到自己电脑或 VPS 上，用配对码绑定账号，不开任何入站端口。
4. 网页 + 手机（PWA）多端同步，断线重连不丢消息。
5. Computer use：bot 能操作节点上的真实浏览器，用户能实时看画面、随时接管。

设计目标（用来验收）

- 非技术用户从零到第一个能对话的 bot：≤ 3 步（注册 → 复制一行命令到服务器 → 选模型）。
- 所有功能在 390px 宽的手机屏幕上可用。
- 用户永远不需要编辑配置文件。

---

2. 整体架构



三条铁律

1. **节点是唯一真相源。**消息、bot、群都存在节点的 SQLite 里；relay 和客户端都只是窗口。
2. **节点不开任何入站端口。**Hermes API 只绑 `127.0.0.1`；所有外部访问都走"节点主动连 relay"的 WebSocket。
3. **不 fork Hermes。**只通过 Hermes 的 CLI 和 API Server 交互，Hermes 升级不应该需要改 nekodot。

---

3. 技术选型（已定，不要替换）

部分	选型
语言 / 运行时	TypeScript（strict），Node.js 24 LTS
包管理	pnpm workspaces 单仓
前端	React 19 + Vite + Tailwind CSS v4 + `motion`（原 Framer Motion）+ Zustand + `vite-plugin-pwa`
远程桌面客户端	`@novnc/novnc`
Relay	Fastify + `ws` + `node:sqlite` + `argon2` + `jose`（JWT）
节点守护进程	纯 Node（`node:sqlite`、`node:child_process`、`ws`），用 esbuild 打成单文件
校验	所有跨进程消息、HTTP 入参都用 `zod` 校验，schema 放在 `packages/protocol`
日志	`pino`
测试	`vitest`（单元）+ Playwright（web 端到端）
国际化	中英双语，`apps/web/src/i18n/{zh,en}.ts`，默认跟随浏览器语言

用 `node:sqlite` 而不是 `better-sqlite3`，是为了让安装脚本不需要编译原生模块。

---

4. 仓库结构

```
nekodot/
├── apps/
│   ├── web/            # React PWA
│   ├── relay/          # 中转服务器（同时托管 web 构建产物和 install.sh）
│   └── node/           # 节点守护进程 nekodot-node
├── packages/
│   ├── protocol/       # zod schema + 类型：信封、RPC、事件、消息
│   └── fake-hermes/    # 测试用假 Hermes：回放录制好的 SSE 事件
├── deploy/
│   ├── install.sh      # 一行安装脚本（由 relay 在 /install.sh 提供）
│   └── docker/         # relay 的 Dockerfile + docker-compose（含 Caddy 自动 HTTPS）
├── docs/
│   ├── hermes-notes.md # Phase 0 核实结论（实施过程中持续更新）
│   └── fixtures/       # 录制的 Hermes SSE 事件
├── design/            # UI 参考稿导出图（见第 11 节）
├── IMPLEMENTATION_PLAN.md
└── README.md
```

5. 节点上的固定约定

项	值
nekodot 主目录	`NEKODOT_HOME=~/.nekodot`
nekodot 专用 Hermes 目录	`HERMES_HOME=~/.nekodot/hermes`（**和用户自己的 `~/.hermes` 完全隔离**）
节点凭证	`~/.nekodot/node.json`：`{ relayUrl, nodeId, nodeToken }`，权限 600
节点数据库	`~/.nekodot/node.db`
日志	`~/.nekodot/logs/`
Hermes API 地址	`http://127.0.0.1:18642`（避开 Hermes 默认的 8642，防止和用户自己的 Hermes 冲突）
Bot 的 profile 名	`nk-` + 6 位小写 base32 随机串，例如 `nk-q7m2xa`（不用显示名，避免中文名和命令别名冲突）
Phase 4 虚拟屏幕	Xvfb `:77`，1440×900；Chromium CDP `127.0.0.1:19222`；x11vnc `127.0.0.1:15900`

每一条 `hermes` 命令都必须带 `HERMES_HOME=~/.nekodot/hermes` 环境变量执行。 封装在 `apps/node/src/hermes/cli.ts` 里，其他地方不准直接 spawn `hermes`。

---

6. 与 Hermes 的集成方式

6.1 Hub profile（默认 profile，不是 bot）

`~/.nekodot/hermes` 下的 default profile 只做一件事：开启多 profile 复用的 API Server。

- `config.yaml`：`gateway.multiplex_profiles: true`
- `.env`：

```
API_SERVER_ENABLED=true
API_SERVER_HOST=127.0.0.1
API_SERVER_PORT=18642
API_SERVER_KEY=<节点首次启动时生成的 32 字节随机 hex>
```

- 不配置任何消息平台（Telegram 等）。
- 节点用子进程启动 `hermes gateway run`（前台模式），退出后按 1s→2s→4s…最多 30s 退避重启。

6.2 Bot = 一个 Hermes profile

创建 bot 时节点依次执行：

1. `hermes profile create nk-xxxxxx`
2. 在 `~/.nekodot/hermes/profiles/nk-xxxxxx/.env` 写入"托管块"（见下），包含该 bot 自己的 `API_SERVER_KEY`（复用模式下每个 profile 必须有自己的 key）和模型 key。
3. `hermes -p nk-xxxxxx config set model.provider …` / `model.default …` /（自定义接口时）`model.base_url …`
4. 写 `SOUL.md` = 用户填的角色描述 + nekodot 托管规则块（见 6.5）。
5. 探测 `GET /p/nk-xxxxxx/health`；如果 404，重启 gateway 子进程后再探测（Phase 0 核实新 profile 是否需要重启才能被服务）。

`.env` 托管块格式（节点只改这两行注释之间的内容，块外内容原样保留）：

```
# >>> nekodot managed >>>
API_SERVER_KEY=...
OPENROUTER_API_KEY=...
# <<< nekodot managed <<<
```

删除 bot：先在 nekodot 数据库里标记 `archived`，然后执行 `hermes profile delete nk-xxxxxx`。

6.3 模型提供方（v1 只支持两种）

1. **OpenRouter**：一个 key 覆盖大量模型。`model.provider=openrouter`，key 写进 `.env` 的 `OPENROUTER_API_KEY`。
2. **自定义 OpenAI 兼容接口**（用户自己平台的模型走这里）：`model.provider=custom`、`model.base_url=<以 /v1 结尾>`、`model.default=<模型 id>`。key 的写法在 Phase 0 核实（候选：`model.api_key` 或由域名推导出的环境变量名），结论写进 `docs/hermes-notes.md`。

提供方配置是**节点级**的（所有 bot 共用一套 key），每个 bot 只选模型 id。模型下拉列表的候选数据来自 `GET /p/<bot>/api/model/options`。

6.4 执行一个回合（Runs API）

所有请求都带 `Authorization: Bearer <该 bot 的 API_SERVER_KEY>`，路径前缀 `/p/<botId>`。

1. `POST /p/<botId>/v1/runs`

```
json

{ "input": "<本回合输入>", "session_id": "<见下>", "instructions": "<可选，群聊上下文>" }
```

请求头带 `Idempotency-Key: <messageId>`，网络重试不会重复执行。返回 `run_id`。 2. `GET /p/<botId>/v1/runs/<run_id>/events`（SSE）。**以 `:` 开头的行是 keepalive，跳过。** 3. 事件到 nekodot 消息的映射：

Hermes 事件	nekodot 动作
`message.delta`	追加到当前 bot 文本消息，通过 `live.delta` 推给客户端（不落库）
`message.interim`	若 `already_streamed=false`，新建一条 `commentary` 消息
`tool.started` `{tool, preview}`	新建 `tool` 消息，状态 `running`
`tool.completed` `{tool, duration, error, preview}`	更新同名、最早一条仍在 `running` 的 tool 消息（FIFO 匹配；Phase 0 核实事件里有没有 call id，有就改用 id 匹配）
`approval.request`	调 `GET /p/<botId>/v1/runs/<run_id>` 取 `pending_controls` 里的 `prompt_id` 和 `execution_generation`，新建 `approval` 消息，并推送 `live.activity` 为 `waiting_approval`
`run.completed` `{output, usage, runtime}`	用 `output` 覆盖最终文本（以它为准），消息状态改为 `done`
`run.failed` / `run.cancelled` / `run.interrupted`	消息状态改为 `failed` / `cancelled`，附错误文案

4. 停止：`POST /p/<botId>/v1/runs/<run_id>/stop`
5. 审批：`POST /p/<botId>/v1/runs/<run_id>/approval`，body 为 `{"request_id": "<prompt_id>", "execution_generation": <n>, "choice": "once" | "deny"}`

session_id 规则

- 私聊：`nk_dm_<conversationId>`
- 群聊：每个 bot 在每个群里一个独立 session，`nk_grp_<groupId>_<botId>`

6.5 SOUL.md 托管规则块（写在用户描述之后，原文照抄）

```
<!-- nekodot managed: do not edit below -->
You are "{displayName}", a member of the user's nekodot team. Your handle is @{handle}.
- Reply in the language the user writes in.
- For passwords, 2FA codes, CAPTCHAs and payments: stop and ask the user to take over the screen. Never type these yourself.
- In group chats, keep replies short. To hand work to a teammate, write @their-handle and say exactly what you need.
<!-- end nekodot managed -->
```

6.6 Phase 0 必须核实的问题（逐条写答案进 `docs/hermes-notes.md`）

1. 官方安装命令（候选：`curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash`，或 GitHub raw 上的 `scripts/install.sh`），以及有没有无交互安装参数。
2. `HERMES_HOME` 指向自定义目录时，`profile create`、`gateway run` 是否都正常。
3. 复用模式下新建 profile 后，是否需要重启 gateway 才能访问 `/p/<id>/...`。
4. 自定义 OpenAI 兼容接口的 key 正确写法；OpenRouter 的环境变量名。
5. `/v1/runs/<id>/events` 的真实事件名和字段（录制一份完整 SSE 到 `docs/fixtures/runs-basic.sse`，再录一份含工具调用和审批的 `runs-tools-approval.sse`）。
6. `tool.started` / `tool.completed` 有没有可用于配对的 id。
7. 浏览器连接自有 Chromium 的配置键（候选：`browser.cdp_url`），以及 bot 用的浏览器工具名。

---

7. 通信协议（`packages/protocol`）

7.1 信封

所有 WebSocket 消息都是 JSON：

```ts
```

```ts
type Envelope = {
  v: 1;
  id: string;          // uuid v4
  type: 'hello' | 'hello.ok' | 'rpc.req' | 'rpc.res' | 'event' | 'live' | 'error' | 'ping' | 'p
  nodeId?: string;     // 客户端发出时必填：要发给哪个节点
  replyTo?: string;    // rpc.res 填对应 rpc.req 的 id
  payload: unknown;    // 按 type 用 zod 校验
};
```

- 连接后 5 秒内必须先发 `hello`（客户端：`{token}`；节点：`{nodeToken, version}`），否则 relay 断开连接。**token 不放在 URL 里。**
- 心跳：relay 每 25 秒发 `ping`，60 秒没收到 `pong` 就断开。
- relay 只检查信封和权限（这个用户是否拥有这个 nodeId），**不解析 payload。**

7.2 RPC（客户端 → 节点）

`rpc.req.payload = { method, params }`；`rpc.res.payload = { ok: true, result } | { ok: false, error: { code, message } }`。超时 30 秒。

method	params	result
`sync.snapshot`	-	`{ bots, groups, conversations, headSeq, node: {platform, computerUse: boolean} }`
`sync.since`	`{ seq }`	`{ events, headSeq, resyncRequired }`（最多 1000 条，超出则 `resyncRequired: true`，客户端重新拉 snapshot）
`messages.list`	`{ conversationId, beforeSeq?, limit≤50 }`	`{ messages }`
`messages.send`	`{ conversationId, text, clientMsgId }`	`{ messageId }`（`clientMsgId` 用于去重）
`runs.stop`	`{ conversationId }`	`{}`（停止该会话当前回合，并清空群聊队列）
`approvals.respond`	`{ messageId, choice: 'once' | 'deny' }`	`{}`
`bots.create`	`{ displayName, handle, avatar, role, model }`	`{ bot }`
`bots.update`	`{ botId, ...可改字段 }`	`{ bot }`
`bots.archive`	`{ botId }`	`{}`
`groups.create`	`{ name, botIds (2–6), leadBotId }`	`{ group }`
`groups.update` / `groups.archive`	…	…
`providers.get` / `providers.set`	`{ kind: 'openrouter' | 'custom', apiKey, baseUrl?, defaultModel? }`	key 永远只返回后 4 位
`models.options`	-	Hermes `/api/model/options` 透传
`screen.status`	-	`{ available, running }`（Phase 4）

7.3 事件（节点→客户端，落库，带 seq）

`event.payload = { seq, kind, data }`，`seq` 是节点全局单调递增整数。`kind` ∈ `bot.upserted` · `bot.archived` · `group.upserted` · `group.archived` · `conversation.upserted` · `message.upserted`

`message.upserted` 携带完整消息对象（不是 diff），客户端按 `messageId` 覆盖即可。

7.4 实时流（节点→客户端，不落库）

- `live.delta { conversationId, messageId, text }`：追加文本。
- `live.activity { conversationId, botId, state: 'thinking' | 'tool' | 'waiting_approval' | 'idle', chain?: { count, max } }`：驱动头像动画和顶栏状态行；群聊里带 `chain`，用于显示"接力已进行 n / 6 轮"。

客户端断线重连后：先 `sync.since(lastSeq)`；对仍是 `streaming` 状态的消息，以 `message.upserted` 里的完整文本为准，再继续接 `live.delta`。

7.5 消息对象

ts

```ts
type Message = {
  messageId: string; conversationId: string; seq: number;
  author: { type: 'user' | 'bot' | 'system'; id?: string };
  kind: 'text' | 'commentary' | 'tool' | 'approval' | 'system';
  status: 'streaming' | 'done' | 'failed' | 'cancelled';
  content:
    | { text: string }                                                    // text / commentary
    | { tool: string; preview: string; resultPreview?: string; durationMs?: number; error?: boo
    | { runId: string; botId: string; requestId: string; executionGeneration: number; prompt: s
  createdAt: string; // ISO
};
```

---

8. 节点数据库（`node:sqlite`）

sql

```
CREATE TABLE bots (
  bot_id TEXT PRIMARY KEY,            -- nk-xxxxxx（= Hermes profile 名）
  display_name TEXT NOT NULL,
  handle TEXT NOT NULL UNIQUE,        -- [a-z0-9-]{1,24}，用于 @
  avatar_json TEXT NOT NULL,          -- {"shape":"dot","color":"#E8603C"}
  role TEXT NOT NULL DEFAULT '',
  model TEXT NOT NULL,
  archived INTEGER NOT NULL DEFAULT 0,
  created_at TEXT NOT NULL
);
CREATE TABLE groups (group_id TEXT PRIMARY KEY, name TEXT NOT NULL, lead_bot_id TEXT NOT NULL,
  archived INTEGER NOT NULL DEFAULT 0, created_at TEXT NOT NULL);
CREATE TABLE group_members (group_id TEXT, bot_id TEXT, position INTEGER, PRIMARY KEY (group_id
CREATE TABLE conversations (conversation_id TEXT PRIMARY KEY, kind TEXT CHECK (kind IN ('dm','g
  bot_id TEXT, group_id TEXT, created_at TEXT NOT NULL);
CREATE TABLE messages (message_id TEXT PRIMARY KEY, conversation_id TEXT NOT NULL, seq INTEGER
  author_type TEXT, author_id TEXT, kind TEXT, status TEXT, content_json TEXT, run_id TEXT,
  client_msg_id TEXT UNIQUE, created_at TEXT NOT NULL);
CREATE TABLE events (seq INTEGER PRIMARY KEY AUTOINCREMENT, kind TEXT NOT NULL, data_json TEXT
  created_at TEXT NOT NULL);
CREATE TABLE settings (key TEXT PRIMARY KEY, value_json TEXT NOT NULL);  -- providers 等
```

规则：任何写 `bots/groups/conversations/messages` 的操作，必须在**同一个事务**里写一条 `events`，并把新 `seq` 写回 `messages.seq`。流式文本每 1 秒或每 400 字落一次库（`message.upserted`），其余时间只发 `live.delta`。

---

9. 群聊编排（精确算法）

1. 用户在群里发消息 → 解析 `@handle`（只认本群成员）。
   - 有 @：按出现顺序，把被 @ 的 bot 依次放进本群队列。
   - 没有 @：把群的 `lead_bot_id` 放进队列。
2. 每个群同一时间只跑一个 bot（串行队列）。
3. 轮到某个 bot 时，`input` 用下面的模板（原文照抄，变量替换）：

```
[nekodot group "{groupName}"] New messages since your last turn:
{每行一条：<显示名> (@handle): <文本>；tool / approval / commentary 消息跳过}

Reply to the group as @{handle}.
```

`instructions` 填："Group members: {所有成员 显示名 (@handle) — role，逗号分隔}. Mention a teammate with @handle only when you need them to act." 4. bot 回复完成后，解析回复里的 `@handle`（排除自己和非成员），追加到队列末尾。 5. 一条用户消息触发的 bot 回合总数上限

`MAX_CHAIN = 6`；达到上限后插入 system 消息"已暂停接力，@某个 bot 继续"，并清空队列。 6. 用户点停止：停止当前 run，清空队列。 7. 回合失败：插入 system 消息，继续执行队列里的下一个。

私聊就是"队列里永远只有一个 bot"的特例，复用同一套代码。

---

10. Relay

HTTP

路由	说明
`POST /api/auth/signup` `{email, password}`	argon2id 哈希；返回 `{token}`（JWT，30 天）
`POST /api/auth/login`	同上；按 IP 每分钟最多 10 次
`GET /api/me`	当前用户 + 名下节点（含在线状态）
`POST /api/pairing-codes`	生成 8 位配对码 `ABCD-2345`（去掉易混字符），10 分钟有效、单次使用；返回 `{code, expiresAt, installCommand}`
`POST /api/pair` `{code, nodeName, platform}`	节点安装时调用；返回 `{nodeId, nodeToken}`（数据库只存 token 的 sha256）
`DELETE /api/nodes/:id`	解绑；立即断开该节点连接
`POST /api/tunnels` `{nodeId, kind: 'vnc'}`	Phase 4：返回 60 秒单次有效的 `ticket`
`GET /install.sh`	返回 `deploy/install.sh`，已替换好 relay 地址
`GET /*`	托管 web 构建产物（SPA fallback）

WebSocket：`/ws/client`、`/ws/node`、`/ws/tunnel?ticket=…`（客户端侧）、`/ws/tunnel-node?tunnelId=…`（节点侧，首条消息带 nodeToken）。

数据库：`users`、`nodes`、`pairing_codes` 三张表。**不存任何聊天内容。**

部署：`deploy/docker/docker-compose.yml` = relay + Caddy（自动 HTTPS）。环境变量：`PUBLIC_URL`、`JWT_SECRET`、`DATA_DIR`。

---

11. UI 与动效规范

参考稿：https://claude.ai/artifact/QzhKbNLbzhQx2q3LvQWrfw （9 张画板：6 个手机页面、桌面三栏、会动的动效规范、颜色/字体/头像）。项目作者需要把每张画板导出成 PNG 放进

`design/`，文件名用画板编号（`01-home.png` … `09-tokens.png`），实施 agent 读 `design/` 里的图。**实现时以本节数值为准**，参考稿只用于看布局和气质；两者冲突时以本节为准。

11.1 气质

温暖、安静、有一点点俏皮。每个 bot 是一个带猫耳的圆点（"neko + dot"）。不要渐变紫、玻璃拟态、霓虹。

11.2 Design tokens（写成 CSS 变量，放 `apps/web/src/styles/tokens.css`）

```css
:root {
  --bg: #FAF7F2;  --surface: #FFFFFF;  --surface-2: #F3EEE6;
  --ink: #1F1B16; --ink-2: #6B645B;    --line: #E8E1D6;
  --accent: #E8603C; --accent-text: #B4432A; --accent-soft: #FCE6DD;
  --ok: #2F9E8F;  --warn: #D9932B;     --danger: #C8432F;
  --bubble-user: #1F1B16; --bubble-user-ink: #FAF7F2;
  --radius-card: 16px; --radius-bubble: 18px; --radius-btn: 12px;
  --font-ui: "Nunito", "PingFang SC", "Noto Sans SC", system-ui, sans-serif;
  --font-mono: "JetBrains Mono", ui-monospace, monospace;
}
@media (prefers-color-scheme: dark) { :root {
  --bg: #16130F; --surface: #211D18; --surface-2: #2A251F;
  --ink: #F3EEE6; --ink-2: #A69E93; --line: #3A342C;
  --accent: #F07A55; --accent-text: #F59A7A; --accent-soft: #3A2620;
  --bubble-user: #F3EEE6; --bubble-user-ink: #16130F;
}}
```

字号：12 / 14（正文）/ 16 / 20 / 28。间距只用 4 的倍数。点击区域最小 44×44。

颜色使用规则（对比度要求，必须遵守）

- 主按钮：`--ink` 底 + `--bg` 字。次按钮：透明底 + 1.5px `--ink` 描边。
- `--accent` 只用于图形、状态点、描边、FAB 底色（FAB 上的图标用 `--ink`）。**不要用 `--accent` 做小字颜色，也不要在 `--accent` 上放白字**（对比度不够）。
- 需要橙色文字（@handle、"需要你"标签、链接）时用 `--accent-text`。
- 停止按钮：白底 + 1px `--danger` 描边 + `--danger` 字。

Bot 头像颜色（8 选 1）：`#E8603C` `#2F9E8F` `#5B7CE6` `#D9932B` `#B05FD0` `#3FA45B` `#E05A8A` `#6B645B`

Bot 头像 SVG（照抄几何，做成 `<BotAvatar color size state />` 组件）

```html
```

```
<svg viewBox="0 0 40 40">
  <polygon class="ear-l" points="7,15 10,3 18,10" fill="{color}"/>
  <polygon class="ear-r" points="33,15 30,3 22,10" fill="{color}"/>
  <circle cx="20" cy="23" r="15" fill="{color}"/>
  <g class="eyes">
    <ellipse cx="15" cy="23" rx="2" ry="2.6" fill="var(--ink)"/>
    <ellipse cx="25" cy="23" rx="2" ry="2.6" fill="var(--ink)"/>
  </g>
</svg>
```

`svg` 设 `overflow: visible`；三个部件都设 `transform-box: fill-box`；左耳 `transform-origin: 100% 100%`，右耳 `0% 100%`，眼睛 `50% 50%`。`state` ∈ `idle`（眨眼）· `working`（耳朵摆 + 眼睛扫视）。品牌 logo = 同一个图形，身体 `--ink`、眼睛 `--accent`。

11.3 页面

页面	手机（<768px）	桌面（≥1024px）
首页	顶部"团队"标题 + 同步状态点和节点名；bot 卡片列表（头像、名字、角色、最后一条消息、工作中三点动画；有待审批或待接管时显示"需要你"标签）；下方"群聊"分组（叠放的成员色点）；右下角 + 按钮（新建 bot / 新建群）	左侧 280px 列表常驻
私聊	顶栏：返回、头像、名字、状态行（"正在回复" / "等你确认"）、屏幕按钮；消息流；bot 工作时输入框上方出现状态条 + 停止按钮	中间栏，最大宽 760px 居中
群聊	同私聊；每条 bot 消息带头像、名字和 @handle；输入 `@` 弹出成员选择；bot 接力时显示"箭头 + 交给 @xxx"标签；接力进行中在消息流底部显示"接力已进行 n / 6 轮"；停止按钮文案为"全部停止"；顶栏副标题显示"主答：<lead 显示名>"	中间栏
新建 bot	底部弹出面板：头像大预览、名字、头像颜色（8 个圆形色块）、"它负责什么？"、模型；"高级设置"折叠：@ 名称；底部主按钮"创建<名字>"	居中弹窗
屏幕	从底部滑出全屏面板（深色）；默认只读，顶部"实时"标记；"接管"后显示"你正在控制 · <bot>已暂停"条 + 呼吸描边；下方显示 bot 的求助消息；底部"键盘" / "只看不操作" / "交还给<bot>"	右侧 480px 深色面板
添加电脑	顶部"第 2 步，共 3 步"；两张卡片："这台电脑" / "云服务器"；两步说明；深色命令框 + "复制命令"；"等待服务器连上…"三点动画 + 配对码倒计时；连上后庆祝动画	居中
设置	模型提供方、电脑列表、语言、退出；"高级"折叠	同

消息组件：用户气泡（深色，靠右）、bot 文本（无气泡，左对齐，头像在左）、commentary（小号、`--ink-2`、斜体）、工具卡片（折叠显示工具名 + 一行 preview + 状态图标，点开看结果 preview，等宽字体）、审批卡片（`--accent-soft` 底 + 1.5px `--accent` 描边，标题"需要你确认"、一句说明、等宽字体的命令、两个按钮"允许一次" / "拒绝"；处理后按钮换成"已允许一次" / "已拒绝"文字）、system 消息（居中小字）。

分阶段显示：顶栏"屏幕"按钮和屏幕面板只在 `sync.snapshot` 的 `node.computerUse=true` 时出现（Phase 4）；新建 bot 时模型下方的能力标签（"支持浏览器操作"等）在 Phase 5 之后才显示，之前只显示提供方名。

11.4 动效（用 `motion` 实现，数值照抄）

名称	触发	规格
消息出现	新消息插入	opacity 0→1，y 8→0，220ms，`cubic-bezier(0.2, 0.8, 0.2, 1)`
流式光标	消息 `streaming`	文末 ▌ 闪烁，1s steps(2) 循环；文本直接追加，不做逐字动画
思考中	`live.activity=thinking`	三个小圆点依次上跳 4px，单个 450ms，错开 120ms，循环
工作中头像	`thinking` 或 `tool`	猫耳交替摆动 ±8°，600ms 循环；眼睛左右扫视，1.6s 循环
头像眨眼	空闲	每 4–7 秒随机一次，眼睛 scaleY 1→0.1→1，160ms
在线状态点	节点在线	外圈 scale 1→1.8、opacity 0.6→0，1.4s 循环
工具卡片展开	点击	高度用 spring（stiffness 400，damping 32）；状态图标 spinner→✓ 交叉淡入 150ms
接力	bot @ 了另一个 bot	"交给 @xxx"标签从左滑入 12px，200ms；被 @ 的头像上跳 4px 再落回，300ms
屏幕面板	打开 / 关闭	手机从底部、桌面从右侧滑入，280ms，同消息曲线
接管中	用户点"接管"	画面描边变成 `--accent` 2px，顶部出现"你正在控制"条；描边呼吸 opacity 0.6↔1，1.2s
配对成功	节点首次连上	新头像 scale 0.6→1.08→1（spring），周围 8 个小圆点向外散开并淡出，600ms
同步状态	断线 / 重连	顶部状态点变 `--warn`，重连中旋转；恢复后变回 `--ok`

`prefers-reduced-motion: reduce` 时：关闭所有循环动画；保留 ≤120ms 的淡入淡出。

12. 分阶段实施

每个 Phase 结束时必须：全部测试通过、`pnpm build` 通过、按"验收"一节的步骤手动走一遍，并在 `docs/progress.md` 记录结果。**一个 Phase 没验收通过，不要开始下一个。**

Phase 0 · 骨架 + 核实 Hermes（约 2–3 天）

- 0.1 建 pnpm 单仓、TS strict、eslint + prettier、vitest、CI（GitHub Actions：lint + test + build）。
- 0.2 `packages/protocol`：把第 7 节的类型全部写成 zod schema，配单元测试（合法/非法样例各一）。
- 0.3 `scripts/spike-hermes.ts`：在临时 `HERMES_HOME` 里按第 6 节流程走一遍（建 hub、建 1 个 bot、配模型、起 gateway、跑一个回合、触发一次工具调用和审批、停止一次），回答 6.6 的 7 个问题，录制 fixtures。
- 0.4 `packages/fake-hermes`：一个本地 HTTP 服务，实现 `/p/:profile/health`、`POST /p/:profile/v1/runs`、`GET …/events`（回放 fixture）、`…/stop`、`…/approval`、`GET …/v1/runs/:id`。之后所有自动化测试都用它，不调用真实模型。

验收：`docs/hermes-notes.md` 7 个问题都有答案；两份 fixture 存在；`pnpm test` 通过。

Phase 1 · 核心闭环：私聊（约 1–1.5 周）

- 1.1 Relay：注册 / 登录、配对码、`/api/pair`、三条 WS 通道、路由转发、心跳。
- 1.2 节点：`nekodot-node pair --relay <url> --code <code>`、`nekodot-node run`；连接 relay（断线退避重连）；hub profile 初始化；gateway 子进程监管；数据库迁移。
- 1.3 节点：`providers.*`、`bots.create/update/archive`、私聊回合（第 6.4 节全部事件映射）、`runs.stop`、`approvals.respond`、`sync.*`、`messages.*`。
- 1.4 Web：登录注册、添加电脑（本地开发时手动跑 pair 命令）、首页 bot 列表、新建 bot、私聊页（流式、工具卡片、审批卡片、停止）、设置页的模型提供方。
- 1.5 Web 状态层：Zustand store 按第 7.4 节实现 snapshot / since / live 合并；断线重连。

验收：本机起 relay + 节点 + web → 注册 → 配对 → 填 OpenRouter key → 建 bot → 对话能流式显示 → 让 bot 执行一个需要审批的命令，点"允许一次"后继续 → 中途点停止能停。用 fake-hermes 的 Playwright 测试覆盖这条路径。

Phase 2 · 群聊（约 1 周）

- 2.1 节点：群 CRUD、第 9 节编排算法、`MAX_CHAIN`、失败继续。
- 2.2 Web：新建群、群聊页、`@` 成员选择器、接力标签、头像工作中动画。
- 2.3 单元测试：用 fake-hermes 覆盖无 @、多 @、bot 互相 @、达到上限、中途停止五种情况。

验收：建一个"研究员 + 写手"群；用户不 @ 时只有 lead 回复；写手 @ 研究员后研究员自动接力；构造互相 @ 的场景，到第 6 回合自动暂停。

Phase 3 · 远程部署 + 多端同步（约 1 周）

- 3.1 `deploy/install.sh`（Linux x64/arm64 + macOS；Windows 只支持 WSL2）：
  1. 检查系统，缺 `curl` / `tar` 就报错退出并说明。
  2. 下载 Node 24 官方二进制到 `~/.nekodot/runtime/`（不用 sudo，不动系统 Node）。
  3. 没有 `hermes` 就运行 Phase 0 确认的官方安装命令。
  4. 从 GitHub Releases 下载 `nekodot-node` 单文件包。
  5. 执行 `nekodot-node pair --relay <内置地址> --code <参数>`。
  6. 注册服务：Linux 用 systemd user service（能用 sudo 就执行 `loginctl enable-linger $USER`）；macOS 用 LaunchAgent。
  7. 最后打印一行"已连接，回到网页继续"。
  - 可以重复执行（幂等）；参数：`--pair <code>`、`--uninstall`。
- 3.2 Web 添加电脑流程：生成配对码 → 显示 `curl -fsSL <relay>/install.sh | bash -s -- --pair ABCD-2345` → 轮询 `/api/me` 等节点上线 → 配对成功动画。
- 3.3 PWA：manifest、图标、离线壳页面；iOS 和 Android 都能"添加到主屏幕"。
- 3.4 多端同步打磨：两台设备同时打开同一会话，一边发消息另一边 1 秒内出现；断网 30 秒恢复后不丢不重。
- 3.5 Relay 的 Docker 部署 + 部署文档。

验收：在一台全新 Ubuntu 24.04 VPS 上只粘贴一行命令完成接入；手机和电脑同时在线、对话实时同步；重启 VPS 后节点自动上线。

Phase 4 · Computer use（浏览器）+ 实时画面（约 1-1.5 周）

- 4.1 节点（仅 Linux）：`ProcessSupervisor` 管理三个子进程——
  - `Xvfb :77 -screen 0 1440x900x24 -nolisten tcp`
  - Chromium（Playwright 安装的版本）：`--remote-debugging-port=19222 --remote-debugging-address=127.0.0.1 --user-data-dir=$NEKODOT_HOME/browser-profile --no-first-run --no-default-browser-check --window-position=0,0 --window-size=1440,900`，环境变量 `DISPLAY=:77`
  - `x11vnc -display :77 -rfbport 15900 -localhost -forever -shared -nopw` install.sh 增加可选步骤 `--with-computer`：apt 安装 `xvfb x11vnc`，并运行 `npx playwright install chromium --with-deps`。
- 4.2 每个 bot profile 设置浏览器连接到 `http://127.0.0.1:19222`（配置键以 Phase 0 结论为准）。
- 4.3 隧道：客户端 `POST /api/tunnels` 拿 ticket → 连 `/ws/tunnel?ticket=` → relay 通知节点 `tunnel.open {tunnelId}` → 节点连 `/ws/tunnel-node` 并 TCP 连 `127.0.0.1:15900` → 双向转发二进制帧。任意一端断开就全部关闭。
- 4.4 Web 屏幕面板：noVNC 默认 `viewOnly=true`。
  - "接管"：如果此时有 bot 的 run 正在执行，先弹确认"接管会停止<bot>当前的操作"，确认后调 `runs.stop`；然后切成可操作，显示"你正在控制 · <bot>已暂停"条和呼吸描

边。
    - "键盘"：聚焦一个隐藏的输入框以唤起手机软键盘，输入转发给 noVNC。
    - "只看不操作"：恢复只读，不给 bot 发任何消息。
    - "交还给<bot>"：恢复只读，并向"打开屏幕面板时所在会话里最后一个发言的 bot"发送回合输入（群聊里同样只发给这一个 bot）："The user finished the manual step on the screen. Continue the task."
- 4.5 macOS / WSL 节点：`screen.status` 返回 `available:false`，UI 显示"电脑操作目前只支持 Linux 节点"。

验收：让 bot "打开 example.com 并告诉我标题"，手机上能实时看到浏览器操作；模拟需要登录的网站，bot 停下请求接管，用户接管输入后交还，bot 继续。

Phase 5 · 全桌面 computer use + 模型兼容表（Phase 4 完成后再细化）

- 在 `:77` 上接入像素级操作的 MCP（截图作为视觉输入，xdotool 执行鼠标键盘；可参考社区项目 hermes-computer-use 的做法），注册为 bot 的 MCP server。
- 建 `docs/model-compat.md`：用 5 个固定任务测试候选模型，标注"支持对话 / 支持浏览器 / 支持全桌面"。UI 选模型时显示这些标签；不支持全桌面的模型自动只启用浏览器工具。

---

13. v1 明确不做

跨用户群聊、端到端加密、原生手机 App、桌面安装包（Electron）、原生 Windows、推送通知、付费 / 计费、通过云厂商 API token 自动开服务器（Phase 3 之后的加分项）、插件商店 UI（直接沿用 Hermes 的 skill / MCP）、每个 bot 独立屏幕（v1 所有 bot 共用一块屏幕）。

---

14. 安全清单（每个 Phase 的 PR 都要自查）

- [ ] Hermes API 只绑 `127.0.0.1`，每个 profile 的 key 都是随机生成的。
- [ ] 节点不监听任何非 loopback 端口。
- [ ] `node.json`、各 profile 的 `.env` 权限都是 600。
- [ ] 模型 key 永远不发回客户端（只回后 4 位），不写进日志。
- [ ] relay 不存聊天内容；日志里不打印 payload。
- [ ] 配对码、隧道 ticket 都是单次使用并有过期时间。
- [ ] 所有 WS / HTTP 入参都经过 zod 校验，不合法直接拒绝。
- [ ] VNC 只能通过 ticket 隧道访问。

---

15. 编码约定

- 禁止 `any`；跨边界的数据一律先过 zod。
- 错误统一为 `{ code: string; message: string }`，`code` 用 `snake_case`，在 `packages/protocol/src/errors.ts` 集中定义。
- 所有面向用户的文案走 i18n，不准写死中文或英文。

- 每个任务一个 commit，commit message 写成 `phase1: 1.3 private chat run mapping` 这种格式。
- 新增依赖要在 PR 描述里说明理由；能用 Node 标准库解决的不加依赖。

---

16. 参考资料（实现时遇到 Hermes 细节先查这里）

- API Server（Runs API、事件、审批、Sessions API、多 profile 路由 `/p/<profile>/`）：https://hermes-agent.nousresearch.com/docs/user-guide/features/api-server
- Profiles（创建、克隆、`config set`、SOUL.md）：https://hermes-agent.nousresearch.com/docs/user-guide/profiles
- 多 profile 复用 gateway（`gateway.multiplex_profiles`）：https://hermes-agent.nousresearch.com/docs/user-guide/multi-profile-gateways
- CLI 命令参考：https://hermes-agent.nousresearch.com/docs/reference/cli-commands
- 浏览器自动化（连接自有 Chromium 的 CDP）：https://hermes-agent.nousresearch.com/docs/user-guide/features/browser
- 参考实现（MIT，可读源码学习做法，不要整包复制）：`NousResearch/Hermes-Bot-Mode`（bot 名册与群聊交互）、`lotsoftick/hermes_client`（Node 后端调用 Hermes、PWA）
- UI 参考稿：见第 11 节开头的链接